

JaxMARL-HFT: GPU-Accelerated Large-Scale Multi-Agent Reinforcement Learning for High-Frequency Trading

Anonymous Author(s)

Abstract

Agent-based modelling (ABM) approaches for high-frequency financial markets are difficult to calibrate and validate in part due to the large parameter space created by defining fixed agent policies. Multi-agent reinforcement learning (MARL) is expected to allow for more realistic agent behaviour and reduce the number of free parameters, but the heavy computational cost of computing experience roll-outs has so far limited research efforts. To address this, we introduce JaxMARL-HFT (*JAX-based Multi-Agent Reinforcement Learning for High-Frequency Trading*), the first GPU-accelerated open-source multi-agent reinforcement learning environment for high-frequency trading (HFT) on market-by-order (MBO) data. Extending the JaxMARL framework, and building on the JAX-LOB implementation, JaxMARL-HFT is designed to handle a heterogeneous set of agents, enabling diverse observation/action spaces and reward functions. It is designed flexibly, so it can also be used for single-agent RL, or extended to act as an ABM with fixed-policy agents. Leveraging JAX allows for a 40x to 240x reduction in end-to-end training time, compared with state-of-the-art reference implementations on the same hardware. This significant speedup makes it feasible to exploit the large, granular datasets available in high-frequency trading, and to perform the extensive hyperparameter sweeps required for robust and efficient MARL research in trading. We demonstrate the use of JaxMARL-HFT with independent Proximal Policy Optimization (IPPO) for a two-player environment, with an order execution and a market-making agent, using one year of LOB data (400 million orders) and show that these learn to outperform standard benchmarks.

1 Introduction

Modelling financial markets at high frequency is an inherently difficult problem, as prices emerge endogenously from the interactions of millions of market participants, often acting with bounded rationality and imperfect information. Agent-based modelling (ABM) addresses this by specifying individual agents and allowing the aggregate market behaviour to arise implicitly from their interactions at the micro-level. In most financial contexts, this amounts to simulating behaviour at the most granular level, i.e. orders submitted to the limit order book (LOB), which is the collection of unmatched orders submitted by market participants. However, as such models are mostly based on pre-defined agent policies, previous approaches have been criticised for having too simplistic agent strategies and emergent behaviour, and sometimes, unfairly, relegated to the realm of “Toy Models”. Reinforcement Learning (RL) has been proposed as a potential solution [2, 12, 30], with most recent work focusing on single-agent RL. To build realistic and robust ABMs with *intelligent* agents – which learn from each other – we require multi-agent reinforcement learning (MARL) which aims to learn independent policies for a heterogeneous agent population. A natural first step

is to successively train RL agents in the single-agent setting, progressively adding their learned policies to the pool of opponent strategies, a form of self-play with differing policy objectives [23].

To allow for learning in an adversarial context, classical MARL has multiple agents independently collecting experiences and simultaneously updating their policies. To the best of our knowledge, there is no open-source framework for this in an HFT context. One reason for this is the computational restrictions and training instabilities common to MARL experiments, exacerbated by the very low signal-to-noise ratio in financial applications. To overcome these challenges, we require a highly efficient simulator to enable large-scale MARL experiments. We therefore present JAXMARL-HFT, the first GPU-enabled MARL environment for HFT on the LOB. Leveraging the power of JAX [6] and extending the parallelisation of JAX-LOB [12] to the multi-agent setting, JaxMARL-HFT is able to achieve up to a 240x reduction in end-to-end training times compared to state-of-the-art simulators. This speed-up allows for more extensive hyperparameter sweeps and for processing tens of thousands of parallel environments, thus unlocking the potential of rich market-by-order (MBO) data sets such as LOBSTER [16]. The environment is based on the JaxMARL framework [28], to allow plug-and-play interactions with state-of-the-art MARL algorithms. It opens up HFT at the most granular level as a complex testbed for evaluating the limits of MARL algorithms.

In section 2 we introduce the core concepts used in this paper, and in section 3 we present an overview of the existing state-of-the-art MARL implementations. Section 4 presents the generalisations necessary to turn a *gymnax* environment [12] into a JaxMARL-compatible [28] MA environment and an overview of the implemented agent archetypes (Section 4.1) and their variations. Section 4.2 considers the changes made to the JaxMARL reference implementation of IPPO. Results for the increased throughput compared to SOTA implementations are discussed in Section 5.1, considering different environment parameters with a major impact on performance. Section 5.2 reports training results and evaluations against baselines of a simple two-player setup. In Section 6, we conclude with a summary of our contributions, and future work.

Our main contributions are summarised as follows:

- (1) Extension of the JAX-LOB [12] environment to the multi-agent setting allowing for **heterogeneous agents**.
- (2) Compatibility with, and extension of, **JaxMARL** [28] algorithm implementations.
- (3) **Major throughput increase** and memory optimisation allowing for training with multiple years of LOBSTER [16] market-by-order data.
- (4) Commitment to **open source** the JaxMARK-HFT framework at the following link [Removed for Double-Blind review].
- (5) A challenging environment with **real-world** applications to test the limits of MARL algorithm development.

(6) Preliminary results showing that, under certain conditions, policies can be learned to outperform baselines, though they require carefully crafted action spaces and reward functions.

2 Background

2.1 High-Frequency Trading

Limit order books (LOBs) [5, 15] are the primary mechanism through which modern financial exchanges operate. As continuous double auction markets, they collect buy and sell limit orders of market participants based on price and time priority, and match compatible orders. Populations of market participants sharing motivations are often classified by trading task. Despite significant intra-class heterogeneity, the formulations of *market making*, *order execution* and *directional trading* prove useful in understanding general population characteristics.

Market making refers to participants who provide liquidity to an exchange by quoting bid and ask prices, aiming to profit from the spread whilst minimising risks associated with accumulating an inventory of securities.

Order execution is a task performed by participants looking to buy or sell a specified quantity of a security over a pre-determined period. The participants in this task aim to minimise the costs associated with this transaction.

Directional trading refers to the general case where a participant aims to profit from short-term price movements by buying and selling a security without the explicit aim for liquidity provision as in the market-making task. These categories provide mathematical frameworks from which control solutions have been designed.

2.2 Multi-Agent Reinforcement Learning (MARL)

Multi-agent reinforcement learning is reinforcement learning where multiple agents learn to act in a common environment simultaneously. The increased computational requirements for training multiple agents simultaneously have typically limited MARL developments. Libraries such as JAX [6] provide a Python interface through which researchers can easily implement hardware acceleration on the GPU and just-in-time (JIT) compilation. Developing MARL environments compatible with JAX acceleration is an active research domain, whilst traditionally environments were designed for the CPU. Rutherford et al. [28] present the JaxMARL library to address this problem; they provide a range of open-source MARL environments leveraging JAX for GPU acceleration.

3 Related Work

Early research in the domain of agent-based modelling for financial markets demonstrates that interactions between basic zero-intelligence agents are sufficient to replicate some key market phenomena [14, 27] and are capable of explaining a substantial share of the cross-sectional variation in bid-ask spreads [11]. However, later studies indicate that capturing certain more nuanced market dynamics, such as price impact [9] or order flow correlation [35], may require agents with more intelligent decision-making capabilities.

More recently, (deep) reinforcement learning has been increasingly applied to different financial trading problems, including in particular directional trading [20, 32], market making [13, 30] and order execution [24, 26]. Some studies have already taken the first

steps towards utilising the powerful learning capabilities of RL agents as a more sophisticated version of agent-based modelling for financial markets by leveraging MARL. Lussange et al. [21] show that a trading simulation with multiple RL agents, based on daily stock-market data, can reproduce several market statistics, specifically price returns, volatility at different horizons, and autocorrelation metrics. In a subsequent study, they support these findings by extending their analysis to daily cryptocurrency closing prices data [22]. Ardon et al. [2] train multiple RL agents that act either as liquidity takers or as liquidity providers, coupled with a stochastic background model following Cont and Muller [8] and demonstrate that groups of agents can learn a wide spectrum of different behaviours. Extending this approach, Yao et al. [37] also employ agents of those two types but remove the background order-book model. Using one day of tick-level order book data from the LOBSTER dataset [16], they show that the resulting simulation reproduces several stylized facts observed in the real data. At the same time, Yao et al. [37] highlight the extensive computational resources required for their training setup. These computational requirements have generally limited the scale of previous MARL research for HFT, or enforced simplified approaches such as stochastic background models instead of data-driven development.

High computational cost and long training times are common problems in MARL applications beyond finance, motivating the development of JaxMARL [28], a JAX-based [6] framework that provides a range of classical MARL algorithms and environments. Our work extends JaxMARL and unlocks the benefits of GPU-accelerated MARL for a real-world-relevant and highly challenging application: modelling financial markets at HFT scales. Based on the JAX-LOB simulator [12], we present a flexible MARL environment that delivers a substantial RL training speed-up. Using GPU acceleration to advance MARL research has been used in application domains beyond the environments presented in JaxMARL, most notably for self-driving, where it has seen significant success [10, 18]. Nevertheless, these examples were not implemented in JAX, but based on an engine [29] written in C++ code compiled to the GPU.

We open source our implementation which places it within the current lineage of publicly available ABM frameworks for high-fidelity LOB trading, such as MAXE [4], ABIDES [7], and PyMarketSim [23]. These frameworks support heterogeneous agents that submit synthetic orders to a simulated limit-order book, enabling controlled studies of market microstructure. ABIDES is complemented by the ABIDES-gym extension [1], which adds an interface for deep reinforcement learning and has already underpinned RL studies on optimal order execution [17, 19] and market making [34]. PyMarketSim [23] also provides a limit-order-market environment populated by a range of heuristic traders and additionally offers training strategic agents with deep reinforcement learning. It also supplies a simplified MARL facility via the Policy-Space Response Oracles procedure, whereby, in each iteration, a single best-response policy is trained via reinforcement learning against the current mixed-strategy equilibrium, then frozen and added to the global strategy pool before the equilibrium is recomputed. Overall, JaxMARL-HFT departs from existing ABM frameworks in two key aspects: it leverages GPU-acceleration for a significant RL training speed-up, and adopts a classical, fully concurrent MARL

paradigm, enabling researchers to use existing algorithm implementations, such as those available in JaxMARL.

4 Design

We design JaxMARL-HFT as a LOB-level MARL environment in JAX [6]. It is built on JAX-LOB [12] and compatible with JaxMARL [28] and follows a classical MARL framework, enabling multiple reinforcement-learning agents to interact both with one another as well as with historical LOB message streams.

Leveraging JAX has three key performance advantages [6, 28]. First, JAX’s vectorized mapping (`vmap`) allows for parallelisation across thousands of GPU threads. Second, the framework benefits from JAX’s just-in-time (JIT) compilation, which automatically fuses operations into an optimized kernel and eliminates Python overhead. Third, as both environment rollouts and learning updates are executed on the GPU, the data-transfer latency arising between CPU and GPU in conventional implementations is removed.

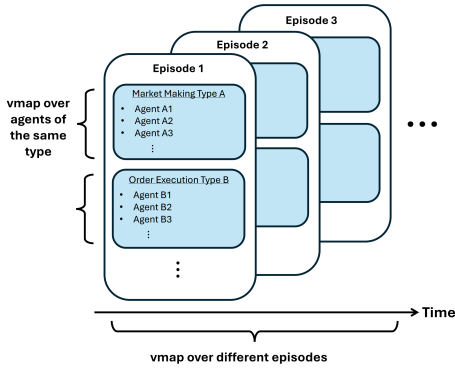


Figure 1: The two levels of parallelisation on the GPU with vectorised mapping (`vmap`) in JAX.

We employ the `vmap` parallelisation in two different ways, as illustrated in Figure 1. First, we partition the dataset into individual episodes. The start and the length of each episode can be chosen flexibly; episodes may even be specified to overlap. At the beginning of each episode, the LOB is initialised as described in Frey et al. [12], using the current state of the book at that time. With `vmap`, we can then process these episodes in parallel. We make a significant improvement compared to JAX-LOB in how messages are loaded on the GPU memory at initialisation. Rather than loading the data in windows, as described in Figure 1 of Frey et al. [12], we load all messages of the dataset in contiguous form and instead keep careful track of the indices where episodes start, and index the data at runtime. This avoids excessive padding and allows a year’s worth of pre-processed AMZN order data to occupy just 4GB of GPU memory, which was not possible in the original implementation.

Second, we parallelise across agents of the same type, at every step within an episode. The reason we do not `vmap` over all agents indiscriminately is JAX-specific: the underlying arrays of the functions (e.g. generating observations or constructing the LOB message based on the action of an agent) must have identical shapes across agents. Since JaxMARL-HFT is designed with the goal of maximum flexibility, allowing for heterogeneous observation spaces, action

spaces and reward functions, this is generally not the case. One possible workaround would be the utilisation of padding, but we aim to keep the possibility for highly diverse agents, e.g. those whose observation spaces only contain a handful of hand-coded features versus those whose observations comprise tokenized messages that could be orders of magnitude larger. In these cases, padding would be extremely inefficient. To preserve flexibility, we iterate in an outer Python for-loop over agent types and `vmap` only over instances of the same type, as depicted in Figure 1. This procedure strikes a good balance: it makes use of the benefits of GPU parallelisation, while maintaining support for heterogeneous agents.

A single environment step is structured as follows:

(1) **Action Conversion:** Agent actions are transformed into LOB messages based on the chosen action space. Unless new messages are at the same price-level, cancellation messages are prepared for unmatched orders from the previous step.

(2) **Random shuffling:** All action messages are randomly shuffled, so there is no ordering in how agents act.

(3) **Augment with market-replay messages:** The cancellation and action messages generated by the agents are concatenated with a background stream of historical messages, in this order. Because the environment is modular, these can be replaced by another background model, with some additional implementation this could be a generative model such as in Nagy et al. [25].

(4) **Processing by JAX-LOB:** Orders are processed by the JAX-LOB simulator as described in [12], and the resulting trades are recorded.

(5) **Calculate step outcomes:** Rewards, auxiliary information, termination flags, and observations are computed. These are used by the agents to determine their next action.

4.1 Different Agent Types

Two heterogeneous agent categories are implemented, covering three HFT tasks, each comprising a set of action spaces, observation spaces, and reward functions.

4.1.1 Market Making. The market-making agent implementation supports three action space configurations, providing a diverse set of strategies and a range of task complexities. The *Spread-Skew* action space is a simplified version of the action space used by Ardon et al. [2], with tabular actions used in place of spread-skew parameters. The *Fixed Quantity* action space is inspired by the work of Spooner et al. [30]. Finally, the *AvSt* action space is a parametrised form of the optimal control solution presented by Avellaneda and Stoikov [3] similar to the implementation in Ardon et al. [2].

In these action spaces, the agent selects bid and ask prices to present to the market from a pre-determined selection of strategies. For example in the *Fixed Quantity* space, the agent has 8 options; i) Not trading; ii/iii) Posting 2 respectively 4 ticks away from the best prices; iv) trading one tick into the spread; v/vi) Posting 2 ticks away from the spread on either side of the book; vii/viii) Posting 5 ticks away from the best price on one side, and 1 tick into the spread on the other.

The quantity associated with each order is fixed a priori through an environment configuration variable, meaning agents following these strategies will quote constant volumes associated with each bid and ask message. In practice, this quantity can be set to the

minimum lot size, which, depending on the asset price, is often the most commonly used trade size by market participants, making this assumption reasonably realistic.

Similar to our approach to action spaces, we design flexible reward functions. We include rewards based on Vadori et al. [33] and Spooner et al. [30], which include *spread* and *inventory* PnL terms, with variable weights given to each term to allow diverse agent risk tolerances. We include several other reward functions, including functions based on portfolio value, and a more complex function based on realised and unrealised PnL. We equip all functions with weighting hyperparameters to present the user with a highly diverse set of possible reward functions.

For the observation space, the market-making agents are presented with a one-dimensional array containing statistics related to the agents’ task, such as the size of their current inventory. We implement multiple similar observation spaces, with a range of complexities, allowing the user to select an observation space with a level of detail appropriate for their use case.

4.1.2 Order Execution. We base the order execution agent on the environment presented by Frey et al. [12]. We discretise the action space, so that rather than selecting a quantity to post at each of the four reference prices, the agent selects only at which price to submit an order of a quantity defined a priori. A more complex version extends this to additionally allow orders with multiples of 2 or 5 of this quantity. The pre-defined order quantity is selected based on the execution task size and episode length.

4.1.3 Directional Trading. Finally, we implement a directional trading action space as a configuration of the market-making class, re-using all reward functions and observation spaces. The similarities between these tasks justify this approach; however, we develop a unique action space to ensure diversity between the classes. The directional trading action space enables the agent to either send a bid or ask order at the best price, or do nothing at each step. All bid and ask messages quote a fixed quantity.

4.2 IPPO and heterogenous agent types

JaxMARL [28] includes MARL algorithm implementations. Given the different classes of agents described previously, we need to extend these implementations to be compatible with our environment. This is relatively easy for an algorithm like independent PPO as we simply maintain separate networks, observations, hidden states, and actions for each agent type, iterating over them during rollout at update steps. Agents of the same or similar types, continue to benefit from efficient batch operations during training as they do for environment rollouts. We opt not to go into great detail on the algorithmic details of IPPO, but highlight that all agents retain separate network weights, and thus only learn about adversarial behaviour from environment observations, not through shared weights.

5 Experimental setup and Results

5.1 Speed Benchmarking

To properly evaluate our environment, we compare it to closely related state-of-the-art implementations. Some of the most similar publicly available frameworks that support multi-agent simulations in HFT are ABIDES-gym [1] and PyMarketSim [23]. Although these implementations permit simulations using multiple heterogeneous

background agents with pre-defined policies, both focus on updating a single reinforcement learning agent; additional agents may interact with the learning agent but do not learn concurrently. Investigating the source-code of ABIDES-gym shows that future support for MARL was planned but does not seem currently supported. PyMarketSim takes a step closer to MARL through its policy-space response oracles, which can be used to train successive RL agents, fixing their policies and adding them to a pool of possible policies, but does not allow for simultaneous learning. We compare to ABIDES-gym and PyMarketSim despite the inexact match as these represent the most similar open-source baselines. Additionally, we implement a CPU version of our MARL environment which we refer to as CPU-MARL; this is to obtain a speed benchmark for order-replay and to quantify the contribution of GPU acceleration.

We use the open-source code of ABIDES-gym and PyMarketSim to compare all implementations on the same hardware. For our experiments, we use a fairly typical compute node for deep learning in academic contexts with up to 8 NVIDIA L40S GPUs and an AMD EPYC 9554 processor with 64 cores and 256 CPU threads available.

5.1.1 Speed of the Environment. Table 1 depicts the environment-step throughput without learning updates. To ensure a fair comparison, we aim to harmonise the dynamics and configuration settings across frameworks, since they differ in several implementation details. In JaxMARL-HFT, in each step both historical market-replay LOBSTER data messages and messages generated by RL agents are processed. For JAXMARL-HFT and CPU-MARL, we consider a different number of agents of two different types: market making and order execution. In the results shown in Table 1, these RL agents choose random actions and no learning updates are performed. PyMarketSim, by contrast, does not use market-replay data. Instead, market activity is generated by background agents (e.g., zero-intelligence agents) that submit orders. We therefore instantiate PyMarketSim with the same total number of zero-intelligence as the sum of data messages and random RL agents in JaxMARL-HFT and CPU-MARL. Because agent arrival times in PyMarketSim follow a geometric distribution, we adjust this distribution so that every agent acts in every step. The simulation using only background agents is not parallelized across CPU cores in PyMarketSim, however their reinforcement learning process - which is based on the tianshou package of Weng et al. [36] - is, and we report a comparison with their parallelized RL pipeline across 64 cores in chapter 5.1.2. For ABIDES-gym, environment step frequency is not defined by number of messages, but by simulation time, we use the rmisc04 reference configuration and empirically adjust the inter-step time to have 1 and 100 orders processed on average. As illustrated in table 1, JAXMARL-HFT achieves a substantial speed-up relative to current state-of-the-art reference implementations and the comparable CPU-based MARL environment, which seems to increase with the number of agents.

5.1.2 Speed of Multi-Agent Reinforcement Learning Training. The key advantages of GPU-acceleration become apparent when considering the multi-agent reinforcement learning training. Unlike current state-of-the-art frameworks, which typically execute environment roll-outs on the CPU and perform policy updates on the GPU, our system carries out both the environment roll-outs and the RL updates entirely on the GPU.

Table 1: Speed comparison between JaxMARL-HFT, PyMarketSim, ABIDES, and CPU-MARL. For JAXMARL-HFT, ABIDES-gym and CPU-MARL we used the RL pipeline with a random policy. We parallelize across 4000 environments on a single GPU for JAXMARL-HFT and 64 CPU cores for ABIDES-gym and CPU-MARL. For PyMarketSim we use their purely zero-intelligence (ZI) agents based simulation on 1 CPU core, matching the number of ZI agents to the sum of of messages and random-policy agents in the other settings.

JaxMARL-HFT							PyMarketSim	ABIDES-gym	CPU-MARL
Envs	Steps	Data Msgs	Types	Agents	Time (s)	Steps/s	Steps/s	Steps/s	Steps/s
4000	50	100	2	1	9.104	21969	463	734	1805
4000	50	1	2	1	0.570	351119	10830	2979	4896
4000	50	100	2	5	13.513	14801	-	-	84
4000	50	1	2	5	2.246	89062	-	-	334
4000	50	100	2	10	18.650	10724	-	-	30
4000	50	1	2	10	4.140	48312	-	-	114

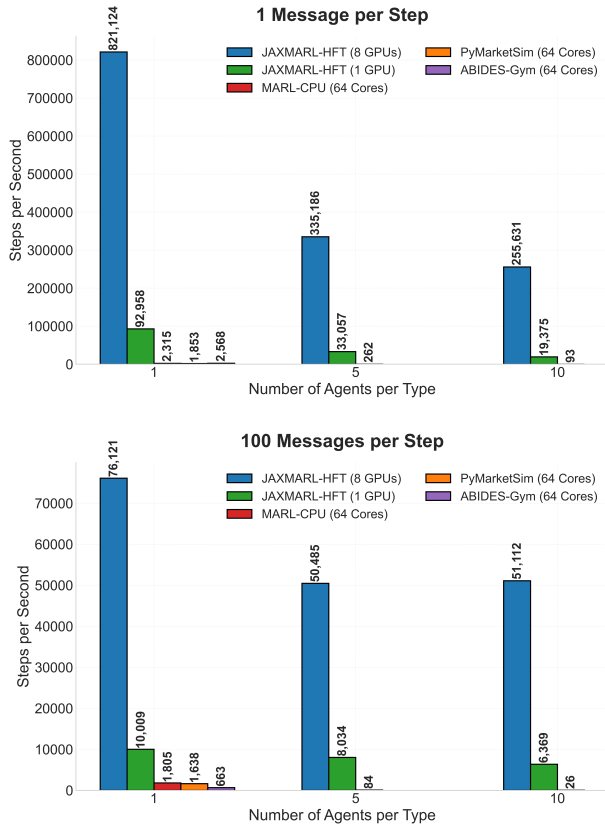


Figure 2: Speed Comparison of the Reinforcement Learning Training Pipeline

With respect to the results in figure 2, it is important to emphasize that, in this comparison, only JAXMARL-HFT and CPU-MARL conduct classical MARL with several learning agents concurrently. By contrast, the results for PyMarketSim and ABIDES-gym each encompass only a single agent updating its policy. We endeavoured to make the comparison as fair as possible. All experiments were run with identical hyper-parameters to ensure comparability:

- Batch size: 20,000

- Experiences collected per update: 200,000
- Epochs per update: 4
- episode length: 50 steps
- Parallel environments: 4,000 for JAXMARL-HFT, 64 for ABIDES-gym, PyMarketSim, CPU-MARL
- Computational resources: All implementations perform their RL updates on an NVIDIA L40S GPU. ABIDES-gym, PyMarketSim, CPU-MARL additionally use an AMD EPYC 9554 processor for their environment rollouts (on 64 cores)

Figure 2 depicts the significant speed advantages of a fully GPU-accelerated framework. The performance benefit becomes particularly pronounced as the number of learning agents increases. While the speed-up for a single agent of each type is approximately 5x - 35x, it rises to 95x - 125x for five agents. For ten agents, we even observe a 200x - 240x speed improvement. Additionally, a JAX-based implementation makes it straightforward to distribute across multiple GPUs through leveraging JAX’s pmap capabilities.

5.2 MARL: Independent PPO for High-Frequency Trading

5.2.1 Training curves and learned policies. Using the adapted IPPO framework (Section 4.2), a two-player environment is considered, containing a Market Making and an Execution agent, learning simultaneously in the MARL environment. An overview of the specifications for each of the agents is given in **ToDo: #1**: Table ???. We use a GRU-based network architecture to ensure recurrence can identify time-dependant patterns in the data. The parameters for each of the networks is as in [12].

The nature and relatively large number of parameters in Table ?? **ToDo: #2**: illustrates one of the open problems with MARL for high-frequency financial applications: stable training is difficult with general-purpose action spaces and multidimensional observation spaces depicting the total state of the LOB. We therefore content ourselves, for the purpose of illustrating the functionality of JaxMARL-HFT, with strongly-simplified action and observation spaces. Multi-modality on the optimal actions, is a reason for which the action spaces are discretised into a relatively small number of distinct classes, rather than allowing more general, continuous

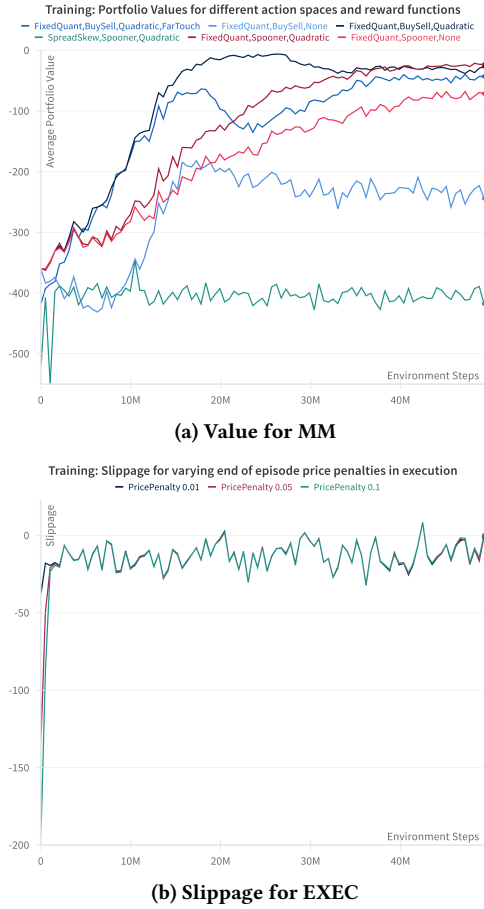


Figure 3: Caption on training curves.

action space described in Frey et al. [12].

Nevertheless, with such settings, we observe some promising results. Figure ?? **ToDo: #3:** shows the training curves for two of the more promising action-spaces. It is important to note that in both cases, we show a proxy value to measure the ability of the agent to execute the given task, which is not necessarily the reward function. We use the *portfolio value* (1) as a measure for the market maker, and the slippage (2) adjusted for the direction of the execution task. This is because it allows for a comparison across different reward functions.

$$PV = Q_{inv} \times P_{mid} + Cash \quad (1)$$

$$Slip = (-1)^{dir} \sum_j Q_j P_j - P_{init} \quad (2)$$

For example, in Figure ?? **ToDo: #4:** we show the differences in training when applying different reward functions: the 'Spooner' reward from Spooner and Savani [31] in red and the 'BuySell' reward based on instantaneous differences of trade prices to the mid-price (??) **ToDo: #5:** in blue.

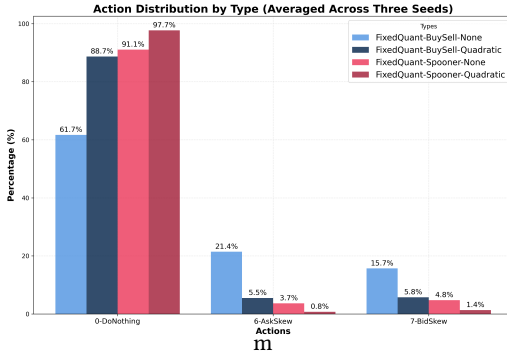
Parameter	Value
Dataset	AMZN 2024
Episode start freq.	Every 64 steps
Episode length	64 steps
Data messages/step	100
JAX-LOB book capacity	100 orders
MM: Action Spaces	[SpreadSkew,FixedQuants]
MM: Reward Functions	[BuySellPnL,Spooner]
MM: Value reference price	[Midprice,Best Price]
MM: Inventory penalty	[Quadratic, None]
MM: Quadratic penalty fact. ρ	50
MM/EXEC: Order size	10
EXEC: Task direction	Uniformly sampled
EXEC: Action Space	Complex (Sec. 4.1.2)
EXEC: Reward func.	See Frey et al. [12]
EXEC: Reward λ	0.0
EXEC: Task size	600
EXEC: Unfilled order penalty	[0.1,0.05,0.05]

Table 2: Key parameters used by the agents considered in Section 5.2, where square brackets indicate multiple options were considered in the presented results.

Further, the effect of a quadratic reward penalty on the held inventory (dark blue/red) is considered. We observe that such a penalty is beneficial, as is the use of the Spooner reward function, which itself depends on the held inventory. It is important to highlight that despite convergence of the loss and reward in both training and validation data, none of the learned strategies for market making are able to make net profits, but instead loose approximately 0.2 ticks on average. Section 5.2.2 will show that this is still a surprisingly good result. It is assumed that one of the hindering factors which makes profitable market making difficult is the fact that posted orders from previous steps are cancelled at each RL step. This is a marked disadvantage in a price-time priority LOB. Addressing thing means re-visiting the environment design and is left for future work.

Figure ?? **ToDo: #6:** a shows the evolution for the execution environment for different coefficients for the end-of-episode penalty if the given amount is not executed successfully. Clearly, a lower value implies a larger reward early on, but the network quickly learns to avoid the costly penalty, after which the difference is negligible.

Digging deeper into the learned policies, we see that the Market Makers **ToDo: #7:** (Figure ?? a) learn to trade very infrequently which explains the convergence towards zero portfolio value in **ToDo: #8:** Figure ?. The only actions which are sampled with non-zero frequency are strong skews with orders deep into one side of the book, and into the spread on the other. One of the drawbacks of this family of reward function is that it they are not



(a) Value for MM



(b) Quant left vs Baseline

Figure 4: Caption on training curves.

normalised by traded volume, as in this case, an optimal policy seems to be to never trade, guaranteeing no losses. Choosing an action space which forces the agent to submit orders at every step is an alternative, but considering the green line in Figure ?? **ToDo: #9:** showing results for the 'SpreadSkew' action space suggests this will result in markedly reduced performance.

For execution, we see that the strategy is generally far more aggressive than the TWAP baseline strategy **ToDo: #10:** (Figure ?? a), likely due to the end-of-episode penalty still being significant. Section 5.2.2 will show, that execution cost can be decreased by up to 20% of a tick as compared to TWAP, likely by trading more passively whilst still guaranteeing execution.

5.2.2 Evaluation against baseline strategies. Table ?? **ToDo: #11:** shows the respective improvements of the learned policies (L) using the *Fixed Quant* action space with the *Spooner* reward function and a quadratic inventory penalty term. The baselines (B) are TWAP in the case of the execution agent and the Avellaneda-Stoikov optimal market making model [3] in the case of the market making agent. Both baselines are outperformed by the learned agent in terms of their respective quality metrics (Portfolio Value and Sippage). Further, the matrix view shows that, at least in the case of the market maker, having a more strategic execution opponent decreases the performance, underlining the expected benefits of MARL when it comes to modelling indirect market impact. We leave an in-depth study of this adversarial behaviour for future work.

Table 3: 'Spooner' reward for for the learned policy after training and baseline policies of each Agent

Market Making	Execution	
	Baseline	Learned
Baseline	Exec: -20.7 MM: -182.8	Exec: -11.3 MM: -182.0
Learned	Exec: -27.9 MM: -11.2	Exec: -12.6 MM: -11.1

Slippage and Portfolio Value: Baseline vs Learned

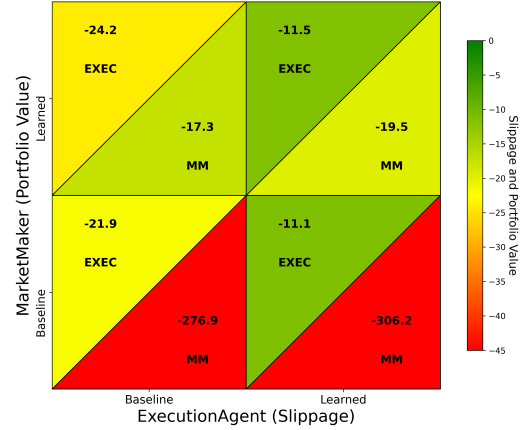


Figure 5: Baseline comparison

6 Conclusion

We present the first framework allowing for highly parallelised, GPU-accelerated MARL experiments for HFT. JaxMARL HFT improves upon the JAX-LOB simulator and extends the single agent *gymnax* environment to be compatible with JaxMARL. The new framework contains heterogeneous implementations of the three main HFT agent tasks: order execution, market making, and directional trading. Compared to similar state-of-the-art implementations, throughput is increased by up to 200x **ToDo: #12:** allowing for larger models, bigger datasets, and more extensive hyperparameter sweeps. Early results show learned policies that out-perform existing baseline policies and show some promising adversarial behaviour.

This comprehensive framework sets the stage for many future research directions:

- Measuring the market impact obtained in simulation with learned agent policies
- Moving to general-purpose action/observation spaces and only parametrizing agent behaviour through the reward function.
- Replacing historical message data with generative models to train RL-agents in adversarial settings, even in the single-agent case.

- Removing historical message data after training and considering the resulting price-series of a multi-agent model with learned agent policies.
- Contrary to order data, simulation allows labelling the origin of submitted orders, allowing for new opportunities in opponent shaping and market participant classification.

References

- [1] Selim Amrouni, Aymeric Moulin, Jared Vann, Svitlana Vyetenko, Tucker Balch, and Manuela Veloso. 2022. ABIDES-gym: gym environments for multi-agent discrete event simulation and application to financial markets. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF '21)*.
- [2] Leo Ardon, Nelson Vadori, Thomas Spooner, Mengda Xu, Jared Vann, and Sumitra Ganesh. 2022. Towards a fully RL-based market simulator. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF '21)*.
- [3] Marco Avellaneda and Sasha Stoikov. 2008. High-frequency trading in a limit order book. *Quantitative Finance* 8, 3 (2008), 217–224.
- [4] Peter Belcak, Jan-Peter Calliess, and Stefan Zohren. 2022. Fast agent-based simulation framework with applications to reinforcement learning and the study of trading latency effects. In *Multi-Agent-Based Simulation XXII*, Koen H. Van Dam and Nicolas Verstaevl (Eds.). Springer International Publishing, Cham, 42–56.
- [5] Jean-Philippe Bouchaud, Julius Bonart, Jonathan Donier, and Martin Gould. 2018. *Trades, quotes and prices: financial markets under the microscope*. Cambridge University Press.
- [6] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. *JAX: composable transformations of Python+NumPy programs*. <http://github.com/google/jax>
- [7] David Byrd, Maria Hybinette, and Tucker Hybinette Balch. 2020. ABIDES: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS '20)*. New York, NY, USA, 11–22.
- [8] Rama Cont and Marvin S Muller. 2021. A stochastic partial differential equation model for limit order book dynamics. *SIAM Journal on Financial Mathematics* 12, 2 (2021), 744–787.
- [9] Wei Cui and Anthony Brabazon. 2012. An agent-based modeling approach to study price impact. In *2012 IEEE Conference on Computational Intelligence for Financial Engineering Economics (CIFER)*.
- [10] Marco Francis Cusumano-Towner et al. 2025. Robust Autonomy Emerges from Self-Play. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=yOXoJpG6Qy>
- [11] J Doyne Farmer, Paolo Patelli, and Ilija I Zovko. 2005. The predictive power of zero intelligence in financial markets. *Proceedings of the National Academy of Sciences* 102, 6 (2005), 2254–2259.
- [12] Sascha Yves Frey, Kang Li, Peer Nagy, Silvia Sapor, Christopher Lu, Stefan Zohren, Jakob Foerster, and Anisoara Calinescu. 2023. JAX-LOB: A GPU-Accelerated limit order book simulator to unlock large scale reinforcement learning for trading. In *Proceedings of the Fourth ACM International Conference on AI in Finance*. 583–591.
- [13] Bruno Gasperov, Stjepan Begušić, Petra Posedel Šimović, and Zvonko Kostanjčar. 2021. Reinforcement learning approaches to optimal market making. *Mathematics* 9, 21 (2021), 2689.
- [14] Dhananjay K. Gode and Shyam Sunder. 1993. Allocative efficiency of markets with zero-intelligence traders: Market as a partial substitute for individual rationality. *Journal of Political Economy* 101, 1 (1993), 119–137. <http://www.jstor.org/stable/2138676>
- [15] Martin D Gould, Mason A Porter, Stacy Williams, Mark McDonald, Daniel J Fenn, and Sam D Howison. 2013. Limit order books. *Quantitative Finance* 13, 11 (2013), 1709–1742.
- [16] Ruihong Huang and Tomas Polak. 2011. Lobster: Limit order book reconstruction system. *Available at SSRN 1977207* (2011).
- [17] Michael Karpe, Jin Fang, Zhongyao Ma, and Chen Wang. 2020. Multi-agent reinforcement learning in a realistic limit order book market simulation. In *Proceedings of the First ACM International Conference on AI in Finance*. ACM, New York New York, 1–7. doi:10.1145/3383455.3422570
- [18] Saman Kazemkhani, Aarav Pandya, Daphne Cornelisse, Brennan Shacklett, and Eugene Vinitisky. 2025. GPUdrive: Data-driven, multi-agent driving simulation at 1 million FPS. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=ERv8ptegFi>
- [19] Siyu Lin and Peter A. Beling. 2021. An agent-based market simulator for back-testing deep reinforcement learning based trade execution strategies. In *Neural Information Processing*. 644–653.
- [20] Yang Liu, Qi Liu, Hongke Zhao, Zhen Pan, and Chuanren Liu. 2020. Adaptive quantitative trading: An imitative deep reinforcement learning approach. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 2128–2135.
- [21] Johann Lussange et al. 2021. Modelling stock markets by multi-agent reinforcement learning. *Computational Economics* 57, 1 (2021), 113–147. doi:10.1007/s10614-020-10038-w
- [22] Johann Lussange, Stefano Vrizzi, Stefano Palminteri, and Boris Gutkin. 2024. Modelling crypto markets by multi-agent reinforcement learning. *arXiv preprint arXiv:2402.10803* (2024).
- [23] Chris Mascioli, Anri Gu, Yongzhao Wang, Mithun Chakraborty, and Michael Wellman. 2024. A financial market simulation environment for trading agents using deep reinforcement learning. In *Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF '24)*. New York, NY, USA, 117–125.
- [24] Ciamac C. Moallemi and Muye Wang. 2022. A reinforcement learning approach to optimal execution. *Quantitative Finance* 22, 6 (2022), 1051–1069. <https://doi.org/10.1080/14697688.2022.2039403>
- [25] Peer Nagy, Sascha Frey, Silvia Sapor, Kang Li, Anisoara Calinescu, Stefan Zohren, and Jakob Foerster. 2023. Generative AI for end-to-end limit order book modelling: A token-level autoregressive generative model of message flow using a deep state space network. In *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF '23)*. 91–99.
- [26] Brian Ning, Franco Ho Ting Lin, and Sebastian Jaimungal. 2021. Double deep q-learning for optimal execution. *Applied Mathematical Finance* 28, 4 (2021), 361–380.
- [27] Richard G Palmer, W Brian Arthur, John H Holland, Blake LeBaron, and Paul T aylor. 1994. Artificial economic life: a simple model of a stockmarket. *Physica D: Nonlinear Phenomena* 75, 1-3 (1994), 264–274.
- [28] Alexander Rutherford et al. 2024. Jaxmarl: Multi-agent rl environments and algorithms in jax. *Advances in Neural Information Processing Systems* 37 (2024), 50925–50951.
- [29] Brennan Shacklett et al. 2023. An Extensible, Data-Oriented Architecture for High-Performance, Many-World Simulation. *ACM Trans. Graph.* 42, 4 (2023).
- [30] Thomas Spooner, John Fearnley, Rahul Savani, and Andreas Koukorinis. 2018. Market making via reinforcement learning. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (Stockholm, Sweden) (AAMAS '18)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 434–442.
- [31] Thomas Spooner and Rahul Savani. 2021. Robust market making via adversarial reinforcement learning. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (Yokohama, Yokohama, Japan) (IJCAI'20)*. Article 633, 7 pages.
- [32] Thibaut Théate and Damien Ernst. 2021. An application of deep reinforcement learning to algorithmic trading. *Expert systems with applications* 173 (2021), 114632.
- [33] Nelson Vadori, Leo Ardon, Sumitra Ganesh, Thomas Spooner, Selim Amrouni, Jared Vann, Mengda Xu, Zeyu Zheng, Tucker Balch, and Manuela Veloso. 2024. Towards multi-agent reinforcement learning-driven over-the-counter market simulations. *Mathematical Finance* 34, 2 (2024), 262–347. doi:10.1111/mafi.12416 _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/mafi.12416>
- [34] Óscar Fernández Vicente, Fernando Fernández Rebollo, and Francisco Javier García Polo. 2022. Deep Q-learning market makers in a multi-agent simulated stock market.
- [35] Svitlana Vyetenko, David Byrd, Nick Petosa, Mahmoud Mahfouz, Danial Derovic, Manuela Veloso, and Tucker Balch. [n. d.]. Get real: realism metrics for robust limit order book market simulations. In *Proceedings of the First ACM International Conference on AI in Finance (ICAIF '20)*.
- [36] Jiayi Weng, Huayu Chen, Dong Yan, Kaichao You, Alexis Duburcq, Minghao Zhang, Yi Su, Hang Su, and Jun Zhu. 2022. Tianshou: A highly modularized deep reinforcement learning library. *Journal of Machine Learning Research* 23, 267 (2022), 1–6.
- [37] Zhiyuan Yao, Zheng Li, Matthew Thomas, and Ionut Florescu. 2024. Reinforcement learning in agent-based market simulation: Unveiling realistic stylized facts and behavior. In *2024 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–9.

Received 3 August 2025